Cheat Sheet de expresiones regulares en JavaScript

Referencia rápida con explicación y ejemplo de uso. Los ejemplos muestran el resultado tras la flecha →.

0. Antes de empezar: cómo leer los ejemplos

Elemento	Qué hace	Ejemplo
`/patrón/flags`	Forma literal de escribir una regex: entre barras y con flags opcionales al final. Es la que usan todos los ejemplos	`/hola/i → regex que busca "hola" sin distinguir mayúsculas`
`new RegExp("patrón", "flags")`	Forma con constructor: el patrón va en un string, así que cada \ se escribe doble. Útil cuando el patrón es dinámico	`new RegExp("\\d+", "g") equivale a /\d+/g`
`→`	No es código: solo indica el resultado que devuelve la expresión de su izquierda. Pruébalo en la consola del navegador (F12) o en Node	`/h.la/.test("hola") → true`
`regex.test(str)`	Comprueba si el texto contiene una coincidencia. Devuelve true o false. Hay que pasarle siempre el texto a evaluar	`/\d/.test("a1") → true`
`regex.exec(str)`	Busca la primera coincidencia. Devuelve un array (posición 0 = coincidencia completa, 1 en adelante = grupos) o null si no hay	`/(\d+)/.exec("ab12") → ["12","12"]`
`str.match(regex)`	Como exec si la regex no lleva g. Con g devuelve un array con todas las coincidencias (sin grupos) o null	`"a1b2".match(/\d/g) → ["1","2"]`
`str.matchAll(regex)`	Devuelve un iterador con todas las coincidencias y sus grupos. La regex debe llevar g	`[..."a1b2".matchAll(/\d/g)] → 2 coincidencias`
`str.replace(regex, rep)`	Devuelve un string nuevo con la primera coincidencia sustituida (todas si lleva g). No modifica el original	`"a-b-c".replace(/-/g, "+") → "a+b+c"`
`str.replaceAll(regex, rep)`	Igual que replace pero sustituye todas; si le pasas una regex, debe llevar g	`"aa".replaceAll(/a/g, "b") → "bb"`
`str.split(regex)`	Divide el texto usando la regex como separador. Devuelve un array de strings	`"a, b;c".split(/[,;]\s*/) → ["a","b","c"]`
`str.search(regex)`	Devuelve el índice de la primera coincidencia, o -1 si no hay. Los índices empiezan en 0	`"hola".search(/l/) → 2`
`[0] [1]`	Acceso a una posición de un array. En match/exec, [0] es la coincidencia completa y [1] el primer grupo de captura	`"ab12".match(/(\d+)/)[1] → "12"`
`.groups`	Objeto con los grupos con nombre (?...). Se accede con .groups.nombre	`"2026-10".match(/(?<anio>\d{4})/).groups.anio → "2026"`
`null`	Lo que devuelven match y exec si no hay coincidencia. Ojo: acceder a [0] sobre null lanza un TypeError	`"abc".match(/\d/) → null`
`[...iterador]`	Spread: convierte un iterador (como el de matchAll) en un array para poder usar .map, .length, etc.	`[..."a1b2".matchAll(/\d/g)].length → 2`

Elemento	Qué hace	Ejemplo
`.map(m => m[0])`	Recorre un array y devuelve otro con el resultado de aplicar la función a cada elemento. m => ... es una función flecha	`[1,2].map(n => n*2) → [2,4]`
`replace con función`	En lugar de un string, replace acepta una función que recibe la coincidencia y devuelve el reemplazo	`"a1".replace(/\d/, n => n*2) → "a2"`
`Ejemplos abreviados (sección 9)`	Para que quepan en la tabla, en los patrones comunes el ejemplo muestra solo .test(...). En código real va tras la regex de la columna izquierda	`/^\d{4}$/.test("2026")` en vez de `.test("2026")`

Un ejemplo completo y ejecutable: `const m = "Pedido 4521".match(/(\d+)/); console.log(m[1]);` muestra 4521 en la consola.

1. Caracteres básicos y clases

Expresión	Explicación	Ejemplo
`.`	Cualquier carácter excepto salto de línea (con flag s también lo incluye)	`/h.la/.test("hola") → true`
`\d`	Un dígito (0-9)	`"a1b2".match(/\d/g) → ["1","2"]`
`\D`	Cualquier carácter que NO sea dígito	`"a1".match(/\D/)[0] → "a"`
`\w`	Letra ASCII, dígito o guion bajo ([A-Za-z0-9_])	`"hi_5!".match(/\w+/)[0] → "hi_5"`
`\W`	Lo contrario de \w	`"hi!".match(/\W/)[0] → "!"`
`\s`	Espacio en blanco (espacio, tab, salto de línea)	`"a b".split(/\s/) → ["a","b"]`
`\S`	Cualquier carácter que NO sea espacio en blanco	`" x ".match(/\S/)[0] → "x"`
`[abc]`	Uno de los caracteres listados	`/[aeiou]/.test("sol") → true`
`[^abc]`	Cualquier carácter excepto los listados	`"abc".match(/[^a]/g) → ["b","c"]`
`[a-z]`	Rango de caracteres	`"Hola".match(/[a-z]/g) → ["o","l","a"]`
`\t \n \r`	Tabulador, salto de línea, retorno de carro	`"a\nb".split(/\n/) → ["a","b"]`
`\. \* \? \\`	Escapa un carácter especial para tratarlo como literal	`/3\.14/.test("3.14") → true`
`\uXXXX`	Carácter Unicode por código hexadecimal	`/\u00f1/.test("año") → true`
`\p{L}`	Cualquier letra Unicode (requiere flag u)	`"ñandú".match(/\p{L}+/u)[0] → "ñandú"`
`\p{Emoji}`	Propiedad Unicode: emojis (requiere flag u)	`/\p{Emoji}/u.test("hola \u{1F600}") → true`

2. Anclas y límites

Expresión	Explicación	Ejemplo
`^`	Inicio de la cadena (o de línea con flag m)	`/^Hola/.test("Hola mundo") → true`
`$`	Fin de la cadena (o de línea con flag m)	`/mundo$/.test("Hola mundo") → true`
`\b`	Límite de palabra	`"un gato".match(/\bgato\b/)[0] → "gato"`
`\B`	Lo contrario de \b (no es límite de palabra)	`/\Bato/.test("gato") → true`

3. Cuantificadores

Expresión	Explicación	Ejemplo
`*`	0 o más veces	`/ab*c/.test("ac") → true`
`+`	1 o más veces	`"aaa".match(/a+/)[0] → "aaa"`
`?`	0 o 1 vez (opcional)	`/colou?r/.test("color") → true`
`{n}`	Exactamente n veces	`/^\d{4}$/.test("2026") → true`
`{n,}`	Al menos n veces	`/\d{3,}/.test("12") → false`
`{n,m}`	Entre n y m veces	`"12345".match(/\d{2,3}/)[0] → "123"`
`*? +? ?? {n,m}?`	Versión perezosa (lazy): coincide con lo mínimo posible	`"<b>a</b><b>b</b>".match(/<b>.*?<\/b>/)[0] → "<b>a</b>"`

4. Grupos y alternancia

Expresión	Explicación	Ejemplo
`(abc)`	Grupo de captura	`"ab12".match(/(\d+)/)[1] → "12"`
`(?:abc)`	Grupo sin captura (solo agrupa)	`/(?:ab)+/.exec("ababx")[0] → "abab"`
`(?<nombre>abc)`	Grupo de captura con nombre	`"2026-10".match(/(?<anio>\d{4})/).groups.anio → "2026"`
`\1, \2…`	Referencia al grupo capturado nº 1, 2…	`/(\w)\1/.test("good") → true`
`\k<nombre>`	Referencia a un grupo con nombre	`/(?<l>\w)\k<l>/.test("hola") → false`
`a|b`	Alternancia: a o b	`/gato|perro/.test("mi perro") → true`

5. Lookaround (aserciones)

Expresión	Explicación	Ejemplo
`X(?=Y)`	Lookahead positivo: X seguido de Y	`"100€".match(/\d+(?=€)/)[0] → "100"`
`X(?!Y)`	Lookahead negativo: X NO seguido de Y	`"1px 2em".match(/\d+(?!px)/)[0] → "2"`
`(?<=Y)X`	Lookbehind positivo: X precedido de Y	`"$50".match(/(?<=\$)\d+/)[0] → "50"`
`(?<!Y)X`	Lookbehind negativo: X NO precedido de Y	`"$1 2".match(/(?<!\$)\b\d/)[0] → "2"`

6. Flags (modificadores)

Expresión	Explicación	Ejemplo
`g`	Global: encuentra todas las coincidencias	`"a1b2".match(/\d/g) → ["1","2"]`
`i`	Ignora mayúsculas/minúsculas	`/hola/i.test("HOLA") → true`
`m`	Multilínea: ^ y $ actúan por línea	`"a\nb".match(/^\w$/gm) → ["a","b"]`
`s`	. también coincide con saltos de línea	`/a.b/s.test("a\nb") → true`
`u`	Modo Unicode (necesario para \p{} y \u{...})	`/\u{1F600}/u.test("\u{1F600}") → true`
`y`	Sticky: busca solo desde lastIndex	`const r=/a/y; r.lastIndex=1; r.test("ba") → true`
`d`	Devuelve índices de inicio/fin de cada grupo (indices)	`/b/d.exec("abc").indices[0] → [1,2]`
`v`	Modo Unicode ampliado (operaciones entre conjuntos)	`/[\p{L}--[a-z]]/v.test("A") → true`

7. Métodos de JavaScript que usan regex

Expresión	Explicación	Ejemplo
`regex.test(str)`	Devuelve true/false si hay coincidencia	`/\d/.test("a1") → true`
`regex.exec(str)`	Devuelve la coincidencia con grupos (o null); con g avanza con lastIndex	`/(\d+)/.exec("ab12")[1] → "12"`
`str.match(regex)`	Sin g: coincidencia con grupos. Con g: array de coincidencias	`"a1b2".match(/\d/g) → ["1","2"]`
`str.matchAll(regex)`	Iterador con todas las coincidencias y sus grupos (exige g)	`[..."a1b2".matchAll(/\d/g)].map(m=>m[0]) → ["1","2"]`
`str.replace(regex, rep)`	Reemplaza la primera coincidencia (o todas con g)	`"a-b-c".replace(/-/g, "+") → "a+b+c"`
`str.replaceAll(regex, rep)`	Reemplaza todas (la regex debe llevar g)	`"aa".replaceAll(/a/g, "b") → "bb"`
`str.split(regex)`	Divide la cadena usando la regex como separador	`"a, b;c".split(/[,;]\s*/) → ["a","b","c"]`
`str.search(regex)`	Índice de la primera coincidencia (o -1)	`"hola".search(/l/) → 2`

8. Patrones especiales en replace

Expresión	Explicación	Ejemplo
`$1, $2…`	Inserta el grupo capturado nº 1, 2…	`"Ana Pérez".replace(/(\w+) (\w+)/, "$2, $1") → "Pérez, Ana"`
`$<nombre>`	Inserta un grupo con nombre	`"2026-10".replace(/(?<a>\d+)-(?<m>\d+)/, "$<m>/$<a>") → "10/2026"`
`$&`	Inserta la coincidencia completa	`"hola".replace(/o/, "[$&]") → "h[o]la"`
`$` ` / `$'`	Texto antes / después de la coincidencia	`"a-b".replace(/-/, "$'") → "abb"`
`$$`	Inserta un símbolo $ literal	`"5".replace(/5/, "$$5") → "$5"`
`(match, ...) => ...`	Función como reemplazo, para lógica personalizada	`"a1".replace(/\d/, n => n*2) → "a2"`

9. Patrones comunes listos para usar

Expresión	Explicación	Ejemplo
`/^[^\s@]+@[^\s@]+\.[^\s@]+$/`	Email (validación simple)	`.test("ana@mail.com") → true`
`/^https?:\/\/[^\s/$.?#].[^\s]*$/i`	URL http/https básica	`.test("https://google.com") → true`
`/^#?([a-f\d]{3}|[a-f\d]{6})$/i`	Color hexadecimal	`.test("#ff0000") → true`
`/^\d{4}-\d{2}-\d{2}$/`	Fecha formato ISO (AAAA-MM-DD, sin validar rangos)	`.test("2026-10-02") → true`
`/^(?:\+34)?[6-9]\d{8}$/`	Teléfono español (móvil/fijo)	`.test("+34612345678") → true`
`/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/`	Contraseña: mín. 8, con minúscula, mayúscula y número	`.test("Hola1234") → true`
`/\s{2,}/g`	Espacios múltiples consecutivos	`"a b".replace(/\s{2,}/g," ") → "a b"`
`/^\s+|\s+$/g`	Espacios al inicio y al final (trim)	`" hola ".replace(/^\s+|\s+$/g,"") → "hola"`
`/\b(\w+)\s+\1\b/gi`	Palabras repetidas seguidas	`/\b(\w+)\s+\1\b/i.test("el el gato") → true`

Expresión	Explicación	Ejemplo
`/[.*+?^${}()|[\]\\]/g`	Escapar caracteres especiales para usar texto como regex	`"a.b".replace(/[.*+?^${}()\|[\]\\]/g,"\\$&") → "a\\.b"`
`/(\d)(?=(\d{3})+$)/g`	Separador de miles	`"1234567".replace(/(\d)(?=(\d{3})+$)/g, "$1.") → "1.234.567"`

Consejos rápidos

- **Escapa el \ en strings:** con `new RegExp("\\d+")` necesitas doble barra; con el literal `/\d+/` no.
- **Cuidado con** `g` **y** `test()`**/**`exec()`**:** una regex con g guarda lastIndex entre llamadas y puede dar resultados alternos. Reinicia con `regex.lastIndex = 0` o evita g si solo validas.
- **Validaciones completas:** usa ^ y $ para que toda la cadena cumpla el patrón, no solo una parte.
- **Evita el backtracking catastrófico:** patrones como `(a+)+$` pueden bloquear el navegador con entradas malintencionadas.
- **Para probar:** regex101.com (selecciona el sabor ECMAScript/JavaScript) o regexr.com.